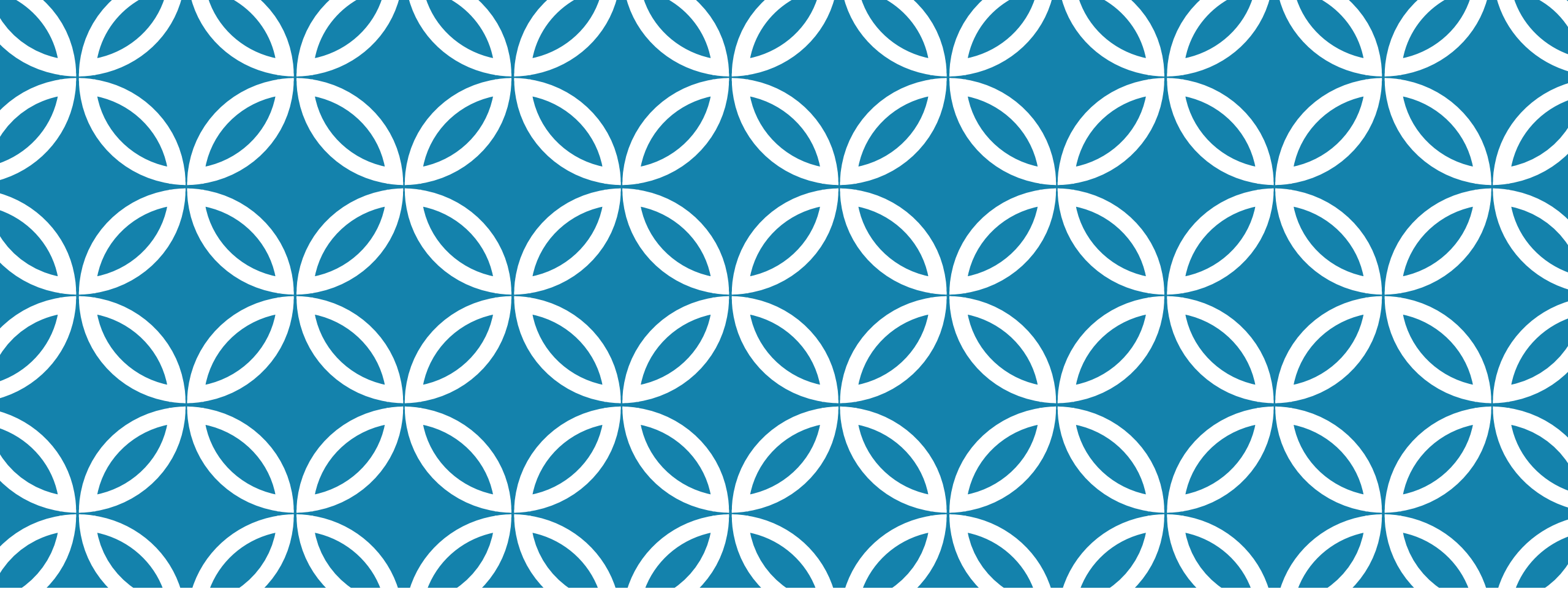




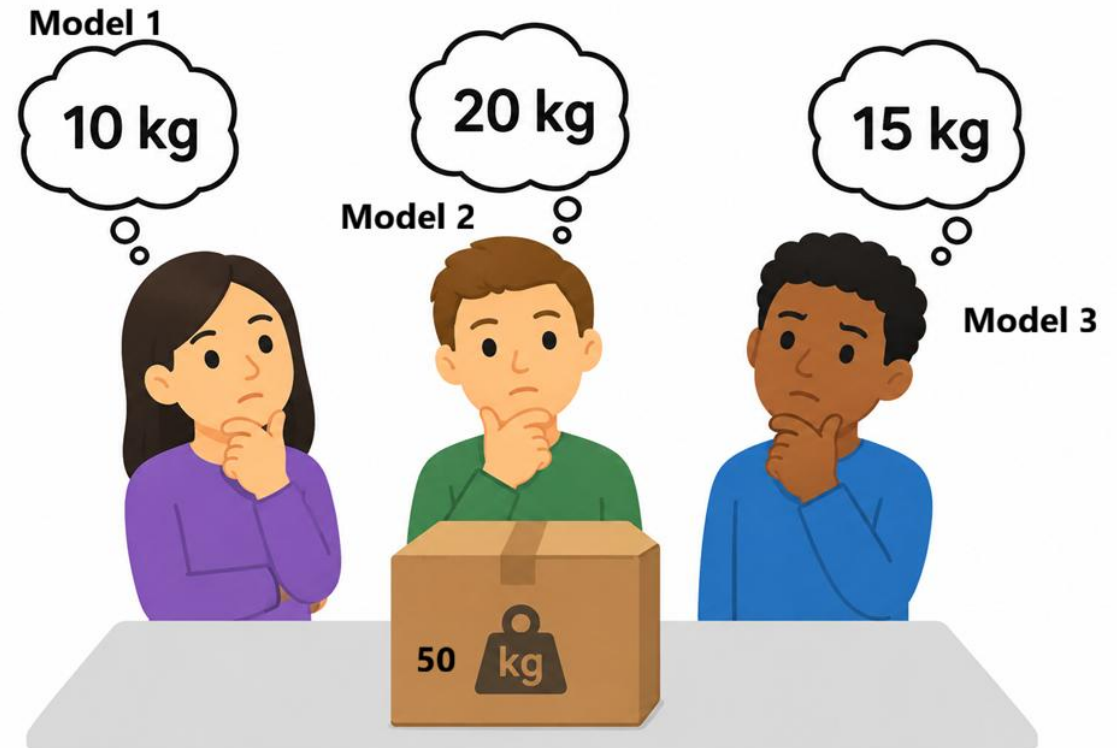
BOOSTING REGRESSOR ALGORITHMS

Sunanda Narasimhan
21-May-2026

WHAT DOES BOOSTING MEAN?

Boosting is an ensemble learning technique that improves prediction accuracy by combining multiple weak learners (i.e. simple models) into a single strong learner (Stronger Model with high prediction score).

Each new model is specifically trained to fix the errors made by the model before it, "boosting" the overall accuracy.



MAJOR PILLARS OF BOOSTING

Weight-Based Boosting (e.g., AdaBoost): It changes the *importance of data points*. It multiplies the importance of misclassified rows so the next model is forced to pay more attention to them.

Gradient-Based Boosting (e.g., GBM, XGBoost, LightGBM): It changes the *target variable*. Instead of predicting the actual outcome, each step predicts the *remaining error* (the gradient of the loss function) left behind by the previous step.

MASTER BLUEPRINT OF BOOSTING ALGORITHM

1 Make a Simple Guess



2 Calculate the Mistakes



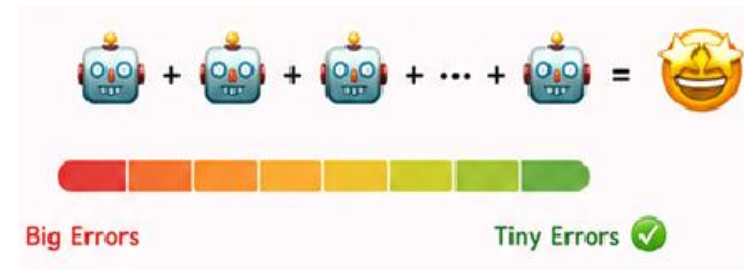
3 Train a New Model on the Mistakes Only



4 Combine and Repeat



5 Repeat Many Times



ADA(ADAPTIVE) BOOSTING

Adaptive: It constantly changes its focus based on what it got wrong in the previous step.

Boosting: It turns a group of average, simple guessers into one absolute genius.

AdaBoost Example: Classifying Cats vs Dogs

Dataset: 6 images (3 Cats 🐱, 3 Dogs 🐶)



1. Initialize weights

All samples start with equal weights.

Image	1	2	3	4	5	6
Weight	1/6	1/6	1/6	1/6	1/6	1/6

2. Train weak learner 1

(e.g., rule: "If ear shape is pointy → Cat, else Dog")

Image	1	2	3	4	5	6
Prediction	Cat	Dog	Cat	Dog	Cat	Cat
Actual	Cat	Dog	Cat	Dog	Cat	Dog
Correct?	✓	✓	✓	✓	✓	✗

$$\text{Error}_1 = 1/6 \approx 0.167$$

$$\text{Learner 1 weight: } \alpha_1 = 0.5 * \ln((1 - 0.167) / 0.167) \approx 0.80$$

3. Update weights

Increase weight of misclassified samples (image 6).

Image	1	2	3	4	6
New Weight	0.1	0.1	0.1	0.1	0.5

(Weights are normalized to sum to 1)

4. Train weak learner 2

(e.g., rule: "If whiskers visible → Cat, else Dog")

Image	1	2	3	4	5	6
Prediction	Cat	Dog	Cat	Dog	Cat	Cat
Actual	Cat	Dog	Cat	Dog	Cat	Dog
Correct?	✓	✓	✓	✓	✓	✗

$$\text{Error}_2 = 1 * 0.5 = 0.5$$

$$\text{Learner 2 weight: } \alpha_2 = 0.5 * \ln((1 - 0.5) / 0.5) = 0$$

5. Update weights again

Increase weight of misclassified samples (image 6).

Image	1	2	3	4	5	6
New Weight	0.083	0.083	0.083	0.083	0.083	0.583

(Weights are normalized to sum to 1)

6. Train weak learner 3

(e.g., rule: "If tail is long → Dog, else Cat")

Image	1	2	3	4	5	6
Prediction	Cat	Dog	Cat	Dog	Cat	Dog
Actual	Cat	Dog	Cat	Dog	Cat	Dog
Correct?	✓	✓	✓	✓	✓	✓

$$\text{Error}_3 = 0$$

$$\text{Learner 3 weight: } \alpha_3 \rightarrow \text{very large (almost infinite)}$$

7. Final strong classifier (weighted vote)

$$H(x) = \text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x))$$

Since learner 3 is perfect, it dominates the vote.



The final model classifies all images correctly by focusing more on the hard example (image 6) in the next rounds.

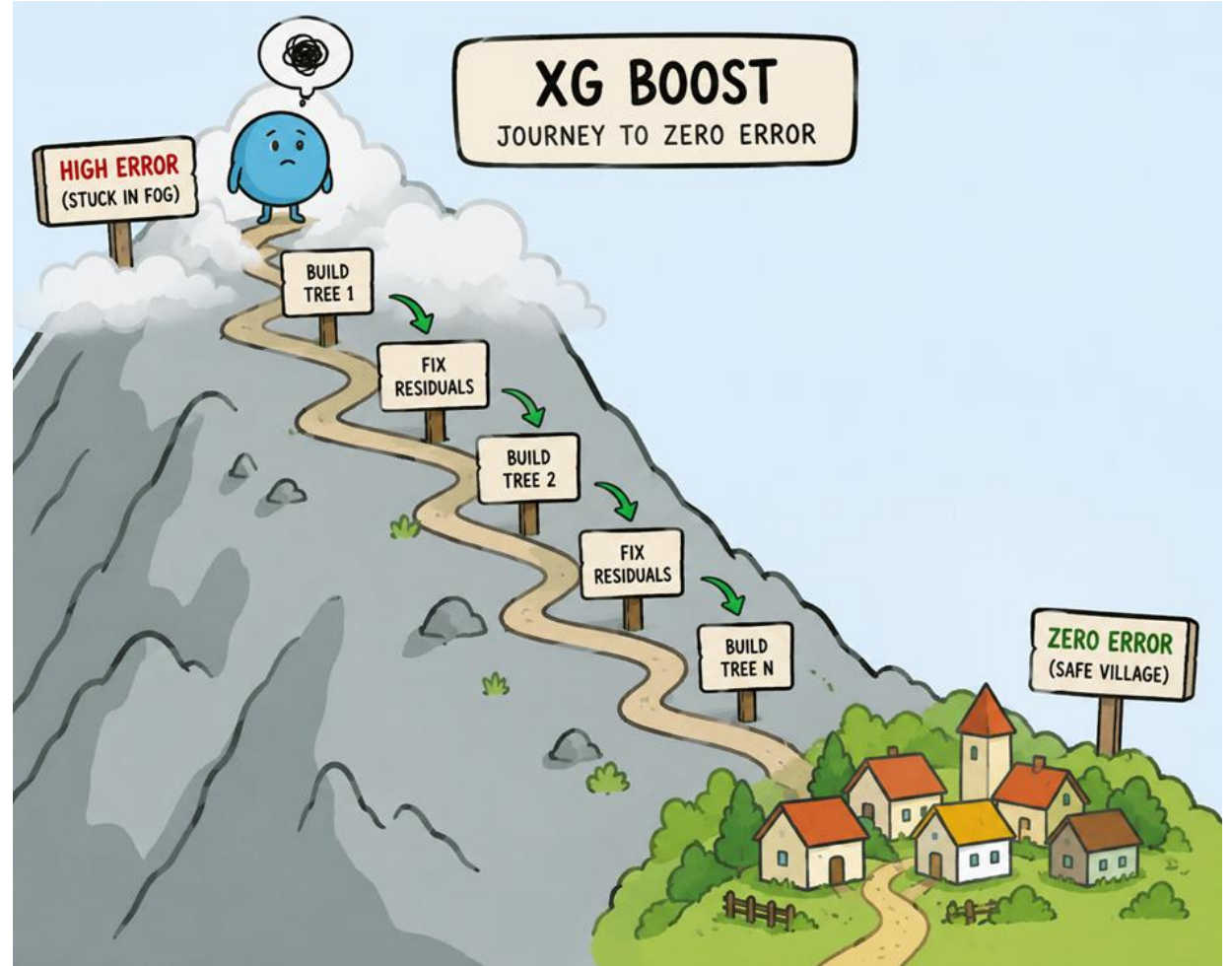
AdaBoost = Train → Focus on mistakes → Combine strong classifier

XGBOOST (EXTREME GRADIENT BOOSTING)

Extreme - it means pushed to the absolute limit, ultra-fast and hyper-efficient

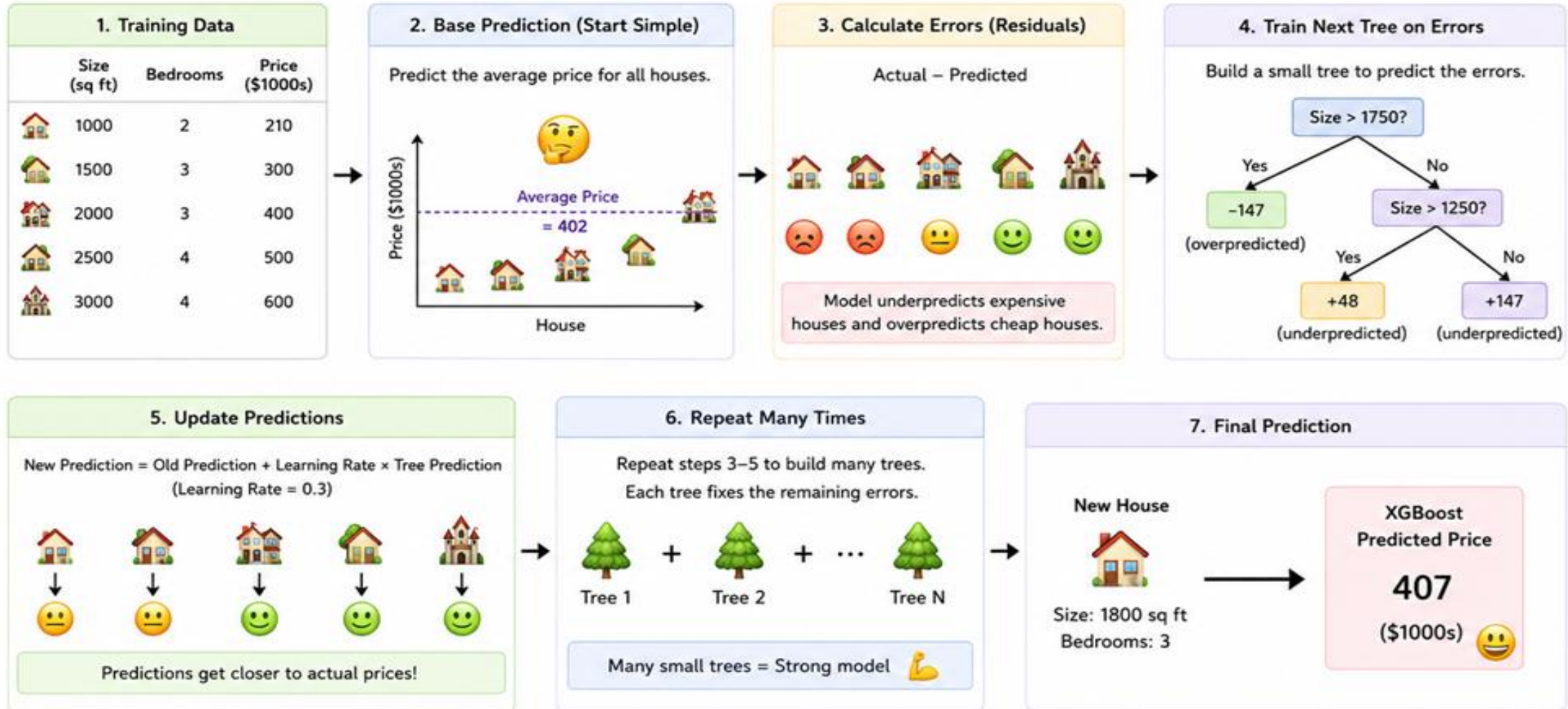
Gradient" means the algorithm calculates exactly *which direction* and *how big* of a step to take to reduce its mistakes.

Boosting Each new tree is specifically designed to predict and fix the leftover errors (called residuals) of the trees that came before it.



XGBoost Example: Predicting House Price 🏠

Goal: Predict house price based on Size and Bedrooms



Why XGBoost Works Well



Many small trees
learn step by step



Focuses on errors
(mistakes)



Regularization avoids
overfitting



Fast and efficient

LIGHT GRADIENT BOOSTING MACHINE(LGBM)

LightGBM (which stands for **Light Gradient Boosting Machine**) is the "Speed Demon" of the machine learning world.

XGBoost gets too slow and heavy when you give them massive amounts of data. While LG groups data into buckets and only fixes the worst errors, it is **lightning fast** and uses **very little RAM**. It allows data scientists to train models on millions of rows of data in seconds rather than hours.

Step 1 : The “**Light**” Part: Grouping Data into Buckets

Step 2: The “**Boosting**” Part: Growing Tall, Not Wide

🌟 LIGHTBGM ALGORITHM 🌟

Sorting thousands of students by exact height 📏

THE PROBLEM

Sorting thousands of students by exact height
(e.g., 160.1 cm, 160.2 cm, 160.5 cm ...)



Testing every micro-difference takes **FOREVER!**

LIGHTBGM SOLUTION 🚀

Instead of checking every tiny difference, LightBGM smartly finds the best sorting by learning from **important patterns!**

1 START SIMPLE

Make a rough guess to split the group.



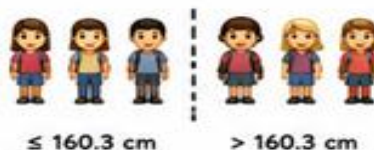
2 LEARN & IMPROVE

Focus on mistakes and tricky students.



3 MAKE BETTER SPLITS

Use what we learned to make smarter splits.



4 REPEAT & REFINE

Keep improving quickly with less effort!



5 PERFECT SORT!

All students are now sorted by exact height!



RESULT

Accurate sorting, super fast! 🚀



💡 LightBGM = Light + Boosting + Gradient-based + Machine Learning Magic 🌟